Local AI RAG configuration system

Alex Huhdanpaa

Purpose:

The purpose of this lab was twofold: first, to become more familiar with the fundamental mechanics of transformer models, and second, to learn how to implement a basic RAG (Retrieval Augmented Generation) system to improve the knowledge accessible through a small to moderate sized AI model.

Background Information:

Modern AI pipelines rely on multiple key tools to provide relevant information in a reliable and efficient manner. One of the most popular tools is Retrieval Augmented Generation (RAG) which involves the combination of development environments, model libraries, LLMs, embedding systems, and vector databases to provide a cohesive system to provide users with repeatable information that works to enhance the reliability and utility of LLM responses.

The foundation of any of these implementations is the development environment. I this case VSCode is being used, it is a lightweight but highly upgradeable environment popular in many development roles due to its flexibility and its integration with various services popular in the industry. It is also notable for its ability to easily test and debug systems allowing for a faster development cycle, however its lightweight nature does lead to various drawbacks, for example the long list of additional installations and downloads required to create a working system that integrates multiple different languages and libraries into a cohesive system.

At the core of the RAG framework as with most NLP frameworks is the Hugging Face Transformers library. It is an open-source framework that provides standardized interfaces for loading and running pretrained transformers used in both inference and vector embedding, simplifying and accelerating the development process. The standardized modular nature of HF transformers allows for rapid testing and experimentation with different models and pipelines through premade offerings from the Hugging Face Hub to build custom setups that are better adapted to their intended role.

One example of a model provided through the Hugging Face Hub is Llama 3.1 8B Instruct. It is a small model originally developed by Meta but finetuned by community efforts. Llama is the family of models designed to provide high quality responses while having a smaller size than other contemporary models. The 8B referred to in the name of the model means it has roughly 8 billion parameters, very small compared to most modern models but this means that it can still be run locally on high end PCs or low end server setups meaning that it is perfect for a project with tight compute limitations compared to larger more modern models that would require expensive custom setups to be able to compare in inference speed with smaller models even if the result is often higher quality. In a RAG setup Llama provides the inference in other words, the generative part based on the user query and the retrieved documents from the RAG pipeline that complement the knowledge the model already has.

The Basic principle underlying a RAG setup is a concept known as embedding, in which a specially trained model will convert a document into a long series of numbers known as an embedding that seek to capture the semantic meaning of the text such that the text's relevance can be searched in a numerical way. One of these such models is Alibaba's Quen3 0.6B embedding, a highly accurate, lightweight embedding model that is able to efficiently and accurately provide the embeddings needed to build a vector database based on semantic similarity. The model also embeds the user query in order to create an embedding that can be used to find semantically similar documents to the query to enable relevant information searching. All this comes together to provide more detailed and accurate information in generated responses.

Once the embeddings are created they must be stored in a way that allows them to efficiently be searched. This is the role of a vector database. One example is HNSWlib which provides a customizable, efficient, and accurate Approximate Nearest Neighbor (ANN) search which allows lower time complexity than a direct comparison search while still providing similar results, the vector database is organized with a Hierarchical Navigable Small World (HNSW) algorithm that roughly works by organizing items into categories with navigation between different documents working based on which neighbor has the closest relation to the embedding of the user query. In a RAG setup a vector database serves as the basis of document storage and retrieval and HNSW is simply a popular method to enable real-time searching of high scalable databases due to its O(ln(n)) time complexity as opposed to O(n).

Retrieval Augmented Generation (RAG) addresses a key limitation of LLMs in that they are unable to access information beyond what they were trained on, thus they often lack domain specific/up to date information which limits how useful they are, RAG addresses this by allowing models to access information outside of their training pool by inputting it directly into the model's context window prior to inference. This both increases the utility of the information while decreasing hallucination where the model makes up conceivable but inaccurate information. There are multiple different ways to build a RAG system where it could either search online for data or it can query a prebuilt database as is the case with what has already been discussed.

Summary:

In this lab I configure a basic RAG setup using Llama 3.1 8B Instruct as the inference model provided by Huggingface Transformers which I used to simplify and standardize my inference pipeline. In addition to this I used Qwen3 0.6B Embedding to create vector embeddings for my searchable documents which were stored in an HNSWlib database to create an efficient, searchable database of documents that can complement the knowledge of the inference model among other benefits. All this was done in python using the above libraries in VSCode which was my IDE. Importantly I also used VSCode's built in virtual environment builder so that if I had any issues with the file I would not have to entirely restart my project.

Codebase:

The Code for this project consists of 5 different python files, being the main.py, what calls each of the classes and functions; llm_runner.py, which handles inference; data_management.py, which handles management of the document set; tensordb.py, which handles the building and searching of the vector database; and finally, config.py, which handles certain global variables and functions.

main.py:

```
import sys
from transformers import AutoTokenizer, LlamaForCausalLM
from tensordb import TensorDB as tdb
from data_management import DataManagement as DM
from llm_runner import LLMRunner
import config

LLMModel = "meta-llama/Llama-3.1-8B-Instruct"

# Tokenizer
```

```
try:
    llm_tokenizer = AutoTokenizer.from_pretrained(LLMModel)
except Exception:
    llm_tokenizer = AutoTokenizer.from_pretrained(LLMModel, use_fast=False)

# Model
try:
    llm_model = LlamaForCausalLM.from_pretrained(
        LLMModel,
        device_map="auto",
        dtype="auto",
        load_in_4bit=True
    )
    try:
        llm_model = llm_model.to("cuda")
    except Exception:
        pass
except Exception:
    llm_model = None
    print("Warning: LLM model failed to load. Generation will be disabled until you configure model
loading.")

def main():
    tensor_db = tdb()
    runner = LLMRunner(tensor_db, llm_model, llm_tokenizer)
    dm = DM(tensor_db)
    while True:
        cmd = input("Optimize, Run, View status, or Quit? (o/r/v/q)\n> ").strip().lower()
        if cmd == "o":
            dm.optimization_input()
        elif cmd == "r":
            runner.run(tensor_db)
        elif cmd == "v":
            dm.view_status()
        elif cmd == "q":
            print("Exiting.")
            sys.exit(0)
        else:
            print("Invalid input.")

if __name__ == "__main__":
    main()
```

llm_runner.py:

```
import torch
import torch.nn.functional as F
from typing import Optional
from transformers import AutoTokenizer, LlamaForCausalLM
```

```
import numpy as np
import config

class LLMRunner:
    """
    Encapsulates generation logic and a `run()` method which queries a TensorDB,
    appends the best result to the prompt, and streams generation token-by-token
    to the same CLI line. The returned decoded final output excludes the prompt.
    """

    def __init__(self, tensor_db, model: Optional[LlamaForCausalLM], tokenizer: AutoTokenizer, device:
Optional[torch.device] = None):
        self.model = model
        self.tokenizer = tokenizer
        self.device = device or self._infer_device()
        self.tensor_db = tensor_db

    def _infer_device(self) -> Optional[torch.device]:
        if self.model is None:
            return None
        # Try model.device, else inspect first parameter
        device = getattr(self.model, "device", None)
        if device is None:
            try:
                device = next(self.model.parameters()).device
            except StopIteration:
                device = None
        return device

    @staticmethod
    def top_k_top_p_filtering(logits: torch.Tensor, top_k: int = 0, top_p: float = 1.0, filter_value: float =
-float('Inf')) -> torch.Tensor:
        """
        Filter a distribution of logits using top-k and/or nucleus (top-p) filtering.
        Operates in-place on the provided logits tensor.
        """
        # Top-k
        if top_k > 0:
            top_k = min(max(top_k, 1), logits.size(-1))
            indices_to_remove = logits < torch.topk(logits, top_k)[0][..., -1, None]
            logits[indices_to_remove] = filter_value

        # Top-p (nucleus)
        if 0.0 < top_p < 1.0:
            sorted_logits, sorted_indices = torch.sort(logits, descending=True)
            cumulative_probs = torch.cumsum(F.softmax(sorted_logits, dim=-1), dim=-1)

            sorted_indices_to_remove = cumulative_probs > top_p
            # shift right to keep first token above threshold
            sorted_indices_to_remove[..., 1:] = sorted_indices_to_remove[..., :-1].clone()
```

```
        sorted_indices_to_remove[..., 0] = False

        indices_to_remove = sorted_indices[sorted_indices_to_remove]
        logits[indices_to_remove] = filter_value

    return logits

def custom_generate(
    self,
    input_ids: torch.Tensor,
    max_new_tokens: int = 256,
    temperature: float = 1.0,
    top_k: int = 50,
    top_p: float = 0.95,
    eos_token_id: Optional[int] = None,
    device: Optional[torch.device] = None
) -> torch.Tensor:
    """
    Autoregressive generator that streams tokens to stdout as they are generated.
    Returns a tensor shaped (1, N_generated) containing only the generated token ids
    (i.e., excludes the input/prompt tokens).
    """
    if self.model is None:
        raise RuntimeError("Model not available for generation.")

    device = device or self.device or input_ids.device
    input_ids = input_ids.to(device)

    batch_size = input_ids.shape[0]
    generated_tokens = []

    past_key_values = None
    cur_input = input_ids

    eos_token_id = eos_token_id or getattr(self.tokenizer, "eos_token_id", None)

    print("\nModel output: ", end="", flush=True)

    with torch.no_grad():
        for step in range(max_new_tokens):
            outputs = self.model(
                input_ids=cur_input,
                past_key_values=past_key_values,
                use_cache=True,
                return_dict=True
            )

            logits = outputs.logits  # (batch, seq_len, vocab)
            past_key_values = getattr(outputs, "past_key_values", None)
```

```
            next_token_logits = logits[:, -1, :].clone()  # (batch, vocab)

            if temperature != 1.0:
                next_token_logits = next_token_logits / max(temperature, 1e-8)

            for b in range(batch_size):
                filtered = self.top_k_top_p_filtering(next_token_logits[b], top_k=top_k, top_p=top_p)
                next_token_logits[b] = filtered

            probs = F.softmax(next_token_logits, dim=-1)
            next_tokens = torch.multinomial(probs, num_samples=1)  # (batch, 1)

            # Save only generated tokens (for batch 0)
            generated_tokens.append(next_tokens[0].item())

            # Stream immediately to stdout
            token_text = self.tokenizer.decode(next_tokens[0].item(), skip_special_tokens=True)
            print(token_text, end="", flush=True)

            # Prepare next input (only last token) for cached forward pass
            cur_input = next_tokens.to(device)

            # Stop if all batches produced EOS
            if eos_token_id is not None:
                if (next_tokens == eos_token_id).all():
                    break

        print("")  # newline after streaming
        return torch.tensor([generated_tokens], device=device)

    def run(self, query_top_k: int = 2, max_new_tokens: int = 256, temperature: float = 0.9, sample_top_k:
int = 10, sample_top_p: float = 0.95):
        """
        Run interaction loop single prompt:
        - Ask user for prompt
        - Query tensor_db for relevant passages
        - Append best passage to prompt and generate using custom_generate()
        - Decode only generated tokens and print final decoded output
        """

        Prompt = input("Please enter your prompt to run the model.\n> ").strip()
        if not Prompt:
            print("Empty prompt.")
            return

        task = "Retrieve relevant documents for the included prompt."
        labels, distances = self.tensor_db.query(query_text = Prompt, topk = 25, instruct_task=task)

        #print(f"Top {query_top_k} results (id: distance):", list(zip(labels, distances)))
        #for ident in labels:
```

```
        #    path = self.tensor_db.get_path_by_id(ident)
        #    text = self.tensor_db.get_text_by_id(ident)
        #    if path:
        #        print(f"--- From file: {path} (id={ident}) ---")
        #    else:
        #        print(f"--- id={ident} ---")
        #    print((text[:1000] + "...") if text and len(text) > 1000 else (text or "[no text stored]"))

        if self.model is None:
            print("LLM model not available locally; skipping generation. If you want generation, load the model.")
            return
        correct_labels = [
            i-1
            for i in labels
        ]
        context_text = self.tensor_db.get_text_by_id(correct_labels[0])
        prompt_to_llm = "Here is the prompt:\n" + Prompt +"\nHere is your context, integrate it in your response if it is helpful in answering:\n" + str(context_text)

        # Tokenize and move to device
        input_ids = self.tokenizer(prompt_to_llm, return_tensors="pt").input_ids.to(self.device)

        gen_ids = self.custom_generate(
            input_ids=input_ids,
            max_new_tokens=max_new_tokens,
            temperature=temperature,
            top_k=sample_top_k,
            top_p=sample_top_p,
            eos_token_id=getattr(self.tokenizer, "eos_token_id", None),
            device=self.device
        )

        # gen_ids contains only generated token ids
        decoded = self.tokenizer.decode(gen_ids[0], skip_special_tokens=True)
```

data_management.py:

```
import torch
import torch.nn.functional as F
from typing import Optional
from transformers import AutoTokenizer, LlamaForCausalLM
import numpy as np
import config

class LLMRunner:
    """
    Encapsulates generation logic and a `run()` method which queries a TensorDB,
    appends the best result to the prompt, and streams generation token-by-token
```

```
    to the same CLI line. The returned decoded final output excludes the prompt.
    """

    def __init__(self, tensor_db, model: Optional[LlamaForCausalLM], tokenizer: AutoTokenizer, device:
Optional[torch.device] = None):
        self.model = model
        self.tokenizer = tokenizer
        self.device = device or self._infer_device()
        self.tensor_db = tensor_db

    def _infer_device(self) -> Optional[torch.device]:
        if self.model is None:
            return None
        # Try model.device, else inspect first parameter
        device = getattr(self.model, "device", None)
        if device is None:
            try:
                device = next(self.model.parameters()).device
            except StopIteration:
                device = None
        return device

    @staticmethod
    def top_k_top_p_filtering(logits: torch.Tensor, top_k: int = 0, top_p: float = 1.0, filter_value: float =
-float('Inf')) -> torch.Tensor:
        """
        Filter a distribution of logits using top-k and/or nucleus (top-p) filtering.
        Operates in-place on the provided logits tensor.
        """
        # Top-k
        if top_k > 0:
            top_k = min(max(top_k, 1), logits.size(-1))
            indices_to_remove = logits < torch.topk(logits, top_k)[0][..., -1, None]
            logits[indices_to_remove] = filter_value

        # Top-p (nucleus)
        if 0.0 < top_p < 1.0:
            sorted_logits, sorted_indices = torch.sort(logits, descending=True)
            cumulative_probs = torch.cumsum(F.softmax(sorted_logits, dim=-1), dim=-1)

            sorted_indices_to_remove = cumulative_probs > top_p
            # shift right to keep first token above threshold
            sorted_indices_to_remove[..., 1:] = sorted_indices_to_remove[..., :-1].clone()
            sorted_indices_to_remove[..., 0] = False

            indices_to_remove = sorted_indices[sorted_indices_to_remove]
            logits[indices_to_remove] = filter_value
```

```
    return logits

def custom_generate(
    self,
    input_ids: torch.Tensor,
    max_new_tokens: int = 256,
    temperature: float = 1.0,
    top_k: int = 50,
    top_p: float = 0.95,
    eos_token_id: Optional[int] = None,
    device: Optional[torch.device] = None
) -> torch.Tensor:
    """
    Autoregressive generator that streams tokens to stdout as they are generated.
    Returns a tensor shaped (1, N_generated) containing only the generated token ids
    (i.e., excludes the input/prompt tokens).
    """
    if self.model is None:
        raise RuntimeError("Model not available for generation.")

    device = device or self.device or input_ids.device
    input_ids = input_ids.to(device)

    batch_size = input_ids.shape[0]
    generated_tokens = []

    past_key_values = None
    cur_input = input_ids

    eos_token_id = eos_token_id or getattr(self.tokenizer, "eos_token_id", None)

    print("\nModel output: ", end="", flush=True)

    with torch.no_grad():
        for step in range(max_new_tokens):
            outputs = self.model(
                input_ids=cur_input,
                past_key_values=past_key_values,
                use_cache=True,
                return_dict=True
            )

            logits = outputs.logits  # (batch, seq_len, vocab)
            past_key_values = getattr(outputs, "past_key_values", None)

            next_token_logits = logits[:, -1, :].clone()  # (batch, vocab)

            if temperature != 1.0:
```

```
            next_token_logits = next_token_logits / max(temperature, 1e-8)

        for b in range(batch_size):
            filtered = self.top_k_top_p_filtering(next_token_logits[b], top_k=top_k, top_p=top_p)
            next_token_logits[b] = filtered

        probs = F.softmax(next_token_logits, dim=-1)
        next_tokens = torch.multinomial(probs, num_samples=1)  # (batch, 1)

        # Save only generated tokens (for batch 0)
        generated_tokens.append(next_tokens[0].item())

        # Stream immediately to stdout
        token_text = self.tokenizer.decode(next_tokens[0].item(), skip_special_tokens=True)
        print(token_text, end="", flush=True)

        # Prepare next input (only last token) for cached forward pass
        cur_input = next_tokens.to(device)

        # Stop if all batches produced EOS
        if eos_token_id is not None:
            if (next_tokens == eos_token_id).all():
                break

    print("")  # newline after streaming
    return torch.tensor([generated_tokens], device=device)

  def run(self, query_top_k: int = 2, max_new_tokens: int = 256, temperature: float = 0.9, sample_top_k:
int = 10, sample_top_p: float = 0.95):
    """
    Run interaction loop single prompt:
    - Ask user for prompt
    - Query tensor_db for relevant passages
    - Append best passage to prompt and generate using custom_generate()
    - Decode only generated tokens and print final decoded output
    """

    Prompt = input("Please enter your prompt to run the model.\n> ").strip()
    if not Prompt:
        print("Empty prompt.")
        return

    task = "Retrieve relevant documents for the included prompt."
    labels, distances = self.tensor_db.query(query_text = Prompt, topk = 25, instruct_task=task)

    #print(f"Top {query_top_k} results (id: distance):", list(zip(labels, distances)))
    #for ident in labels:
    #    path = self.tensor_db.get_path_by_id(ident)
```

```
    #    text = self.tensor_db.get_text_by_id(ident)
    #    if path:
    #        print(f"--- From file: {path} (id={ident}) ---")
    #    else:
    #        print(f"--- id={ident} ---")
    #    print((text[:1000] + "...") if text and len(text) > 1000 else (text or "[no text stored]"))

    if self.model is None:
        print("LLM model not available locally; skipping generation. If you want generation, load the
model.")
        return
    correct_labels = [
        i-1
        for i in labels
    ]
    context_text = self.tensor_db.get_text_by_id(correct_labels[0])
    prompt_to_llm = "Here is the prompt:\n" + Prompt +"\nHere is your context, integrate it in your
response if it is helpful in answering:\n" + str(context_text)

    # Tokenize and move to device
    input_ids = self.tokenizer(prompt_to_llm, return_tensors="pt").input_ids.to(self.device)

    gen_ids = self.custom_generate(
        input_ids=input_ids,
        max_new_tokens=max_new_tokens,
        temperature=temperature,
        top_k=sample_top_k,
        top_p=sample_top_p,
        eos_token_id=getattr(self.tokenizer, "eos_token_id", None),
        device=self.device
    )

    # gen_ids contains only generated token ids
    decoded = self.tokenizer.decode(gen_ids[0], skip_special_tokens=True)
```

tensordb.py:

```
import torch
import torch.nn.functional as F
from typing import Optional
from transformers import AutoTokenizer, LlamaForCausalLM
import numpy as np
import config

class LLMRunner:
    """
    Encapsulates generation logic and a `run()` method which queries a TensorDB,
```

```
    appends the best result to the prompt, and streams generation token-by-token
    to the same CLI line. The returned decoded final output excludes the prompt.
    """

    def __init__(self, tensor_db, model: Optional[LlamaForCausalLM], tokenizer: AutoTokenizer, device:
Optional[torch.device] = None):
        self.model = model
        self.tokenizer = tokenizer
        self.device = device or self._infer_device()
        self.tensor_db = tensor_db

    def _infer_device(self) -> Optional[torch.device]:
        if self.model is None:
            return None
        # Try model.device, else inspect first parameter
        device = getattr(self.model, "device", None)
        if device is None:
            try:
                device = next(self.model.parameters()).device
            except StopIteration:
                device = None
        return device

    @staticmethod
    def top_k_top_p_filtering(logits: torch.Tensor, top_k: int = 0, top_p: float = 1.0, filter_value: float =
-float('Inf')) -> torch.Tensor:
        """
        Filter a distribution of logits using top-k and/or nucleus (top-p) filtering.
        Operates in-place on the provided logits tensor.
        """
        # Top-k
        if top_k > 0:
            top_k = min(max(top_k, 1), logits.size(-1))
            indices_to_remove = logits < torch.topk(logits, top_k)[0][..., -1, None]
            logits[indices_to_remove] = filter_value

        # Top-p (nucleus)
        if 0.0 < top_p < 1.0:
            sorted_logits, sorted_indices = torch.sort(logits, descending=True)
            cumulative_probs = torch.cumsum(F.softmax(sorted_logits, dim=-1), dim=-1)

            sorted_indices_to_remove = cumulative_probs > top_p
            # shift right to keep first token above threshold
            sorted_indices_to_remove[..., 1:] = sorted_indices_to_remove[..., :-1].clone()
            sorted_indices_to_remove[..., 0] = False

            indices_to_remove = sorted_indices[sorted_indices_to_remove]
            logits[indices_to_remove] = filter_value
```

```
    return logits

def custom_generate(
    self,
    input_ids: torch.Tensor,
    max_new_tokens: int = 256,
    temperature: float = 1.0,
    top_k: int = 50,
    top_p: float = 0.95,
    eos_token_id: Optional[int] = None,
    device: Optional[torch.device] = None
) -> torch.Tensor:
    """
    Autoregressive generator that streams tokens to stdout as they are generated.
    Returns a tensor shaped (1, N_generated) containing only the generated token ids
    (i.e., excludes the input/prompt tokens).
    """
    if self.model is None:
        raise RuntimeError("Model not available for generation.")

    device = device or self.device or input_ids.device
    input_ids = input_ids.to(device)

    batch_size = input_ids.shape[0]
    generated_tokens = []

    past_key_values = None
    cur_input = input_ids

    eos_token_id = eos_token_id or getattr(self.tokenizer, "eos_token_id", None)

    print("\nModel output: ", end="", flush=True)

    with torch.no_grad():
        for step in range(max_new_tokens):
            outputs = self.model(
                input_ids=cur_input,
                past_key_values=past_key_values,
                use_cache=True,
                return_dict=True
            )

            logits = outputs.logits  # (batch, seq_len, vocab)
            past_key_values = getattr(outputs, "past_key_values", None)

            next_token_logits = logits[:, -1, :].clone()  # (batch, vocab)
```

```
            if temperature != 1.0:
                next_token_logits = next_token_logits / max(temperature, 1e-8)

            for b in range(batch_size):
                filtered = self.top_k_top_p_filtering(next_token_logits[b], top_k=top_k, top_p=top_p)
                next_token_logits[b] = filtered

            probs = F.softmax(next_token_logits, dim=-1)
            next_tokens = torch.multinomial(probs, num_samples=1)  # (batch, 1)

            # Save only generated tokens (for batch 0)
            generated_tokens.append(next_tokens[0].item())

            # Stream immediately to stdout
            token_text = self.tokenizer.decode(next_tokens[0].item(), skip_special_tokens=True)
            print(token_text, end="", flush=True)

            # Prepare next input (only last token) for cached forward pass
            cur_input = next_tokens.to(device)

            # Stop if all batches produced EOS
            if eos_token_id is not None:
                if (next_tokens == eos_token_id).all():
                    break

        print("")  # newline after streaming
        return torch.tensor([generated_tokens], device=device)

    def run(self, query_top_k: int = 2, max_new_tokens: int = 256, temperature: float = 0.9, sample_top_k:
int = 10, sample_top_p: float = 0.95):
        """
        Run interaction loop single prompt:
        - Ask user for prompt
        - Query tensor_db for relevant passages
        - Append best passage to prompt and generate using custom_generate()
        - Decode only generated tokens and print final decoded output
        """

        Prompt = input("Please enter your prompt to run the model.\n> ").strip()
        if not Prompt:
            print("Empty prompt.")
            return

        task = "Retrieve relevant documents for the included prompt."
        labels, distances = self.tensor_db.query(query_text = Prompt, topk = 25, instruct_task=task)

        #print(f"Top {query_top_k} results (id: distance):", list(zip(labels, distances)))
        #for ident in labels:
```

```
    #    path = self.tensor_db.get_path_by_id(ident)
    #    text = self.tensor_db.get_text_by_id(ident)
    #    if path:
    #        print(f"--- From file: {path} (id={ident}) ---")
    #    else:
    #        print(f"--- id={ident} ---")
    #    print((text[:1000] + "...") if text and len(text) > 1000 else (text or "[no text stored]"))

    if self.model is None:
        print("LLM model not available locally; skipping generation. If you want generation, load the
model.")
        return
    correct_labels = [
        i-1
        for i in labels
    ]
    context_text = self.tensor_db.get_text_by_id(correct_labels[0])
    prompt_to_llm = "Here is the prompt:\n" + Prompt +"\nHere is your context, integrate it in your
response if it is helpful in answering:\n" + str(context_text)

    # Tokenize and move to device
    input_ids = self.tokenizer(prompt_to_llm, return_tensors="pt").input_ids.to(self.device)

    gen_ids = self.custom_generate(
        input_ids=input_ids,
        max_new_tokens=max_new_tokens,
        temperature=temperature,
        top_k=sample_top_k,
        top_p=sample_top_p,
        eos_token_id=getattr(self.tokenizer, "eos_token_id", None),
        device=self.device
    )

    # gen_ids contains only generated token ids
    decoded = self.tokenizer.decode(gen_ids[0], skip_special_tokens=True)
```

config.py:

```
# Shared variables

texts = []
txt_files = []
```

There was also a requirements.txt file that had all of the libraries and dependencies that were ever downloaded at some point, so do acknowledge that many of these are not needed for the program to run, finally there was a README file to get new users started

requirements.txt:

```
accelerate==1.11.0
aiohappyeyeballs==2.6.1
aiohttp==3.13.2
aiosignal==1.4.0
annotated-types==0.7.0
anyio==4.11.0
attrs==25.4.0
bitsandbytes==0.48.2
blis==1.3.2
catalogue==2.0.10
certifi==2025.11.12
charset-normalizer==3.4.4
click==8.3.1
cloudpathlib==0.23.0
colorama==0.4.6
confection==0.1.5
curated-tokenizers==0.0.9
curated-transformers==0.1.1
cymem==2.0.13
datasets==4.4.1
dill==0.4.0
en_core_web_lg @
https://github.com/explosion/spacy-models/releases/download/en_core_web_lg-3.8.0/
en_core_web_lg-3.8.0-py3-none-
any.whl#sha256=293e9547a655b25499198ab15a525b05b9407a75f10255e405e8c3854329ab63
en_core_web_sm @
https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/
en_core_web_sm-3.8.0-py3-none-
any.whl#sha256=1932429db727d4bff3deed6b34cfc05df17794f4a52eeb26cf8928f7c1a0fb85
en_core_web_trf @
https://github.com/explosion/spacy-models/releases/download/en_core_web_trf-3.8.0/
en_core_web_trf-3.8.0-py3-none-
any.whl#sha256=272a31e9d8530d1e075351d30a462d7e80e31da23574f1b274e200f3fff35bf5
filelock==3.19.1
frozenlist==1.8.0
fsspec==2025.9.0
grpcio==1.76.0
h11==0.16.0
hnswlib==0.8.0
httpcore==1.0.9
httpx==0.28.1
huggingface-hub==0.36.0
idna==3.11
Jinja2==3.1.6
lxml==6.0.2
```

MarkupSafe==2.1.5
milvus==2.2.16
mpmath==1.3.0
multidict==6.7.0
multiprocess==0.70.18
murmurhash==1.0.15
networkx==3.5
numpy==2.3.3
orjson==3.11.4
packaging==25.0
pandas==2.3.3
pillow==11.3.0
preshed==3.0.11
propcache==0.4.1
protobuf==6.33.1
psutil==7.1.3
pyarrow==22.0.0
pydantic==2.12.4
pydantic_core==2.41.5
pymilvus==2.6.3
python-dateutil==2.9.0.post0
python-dotenv==1.2.1
pytz==2025.2
PyYAML==6.0.3
regex==2025.11.3
requests==2.32.5
rustenv==0.0.6
safetensors==0.6.2
setuptools==70.2.0
six==1.17.0
smart_open==7.5.0
sniffio==1.3.1
spacy==3.8.11
spacy-curated-transformers==0.3.1
spacy-legacy==3.0.12
spacy-loggers==1.0.5
srsly==2.5.1
sympy==1.14.0
thinc==8.3.9
tokenizers==0.22.1
torch==2.9.1+cu130
torchvision==0.24.1+cu130
tqdm==4.67.1
transformers==4.57.1
typer-slim==0.20.0
typing-inspection==0.4.2
typing_extensions==4.15.0
tzdata==2025.2

urllib3==2.5.0
wasabi==1.1.3
weasel==0.4.3
wrapt==2.0.1
xxhash==3.6.0
yarl==1.22.0

README.txt:

How to run program:

Downloads:
1. Download VScode and install python

Linux terminal commands:
1. sudo apt-get update
2. sudo apt-get install build-essential gdb

VScode:
1. Open a file and select project root directory
2. CRTL+Shift+P
3. Type Python: Create Environment
4. Select Venv from list of environment types

Install required libraries:
pip install -r requirements.txt

OR

pip install sys
pip install transformers
pip install torch
pip install numpy
pip install typing
pip install torch.nn.functional
pip install os
pip install hnswlib

Problems:

This project was never going to be easy given my long hiatus from programming to focus on other projects. This meant that I had to relearn a lot of critical python commands and programming principles such as the structure of classes, functions, and objects in OOPLs (Object Oriented Programming Languages) such as python. The first issue I

encountered was when I found a finetuned model that would suit my project goals, Llama 3.1 8B Instruct offered as a community project on HuggingFace. However this came with the caveat that I had to apply for approval to use the model, which took a couple days to get meaning my project could not continue. My next issue was when I attempted to install Pytorch, a critical library for managing the tensors used by LLMs to store embeddings during inference. This library uses C and needs C build tools installed to create a working tensor, and due to VSCode's lightweight nature these tools do not come preinstalled to save size, this meant that I had to install them to use this critical library. The next issue was when I could not figure out how to make the HNSW search refer to the documents relevant to the search although I found this was due to trying to maximize as opposed to minimize the distance to the document in the database. Following this I had an issue where I confused the topK of the prediction of the next word at the end of the inference cycle with the topK of the vectorDB search meaning that rather than changing the number of referenced documents returned to be added to the user query sent to the inference model I was changing how mathematically inaccurate of words I was willing to allow my model to use resulting in a response that didn't match well to the query due to cumulative errors caused by the nature of the recursive nature of LLMs. My final issue was that when I tried to clone my repository from one computer to another I found that the program didn't run, although this was an easy fix as I just added a file that the user can run to install all of the dependencies prior to use making it a simple one command install.

Screenshots:

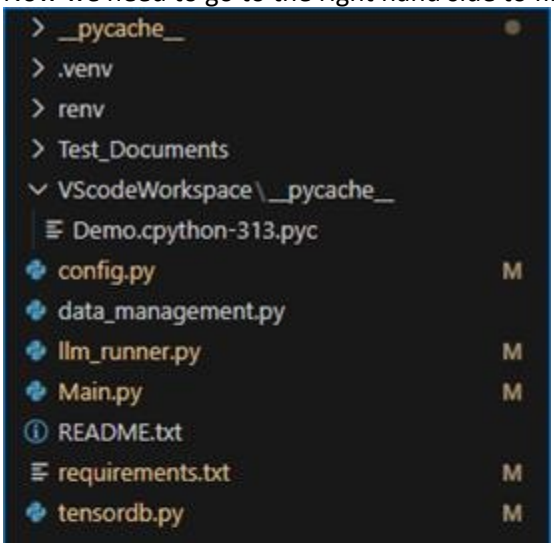
The Process started by running the main.py file with the run button in the VSCode IDE. As you can see in the terminal this causes the program to start and the model to begin loading into the GPU

As you can see in the GPU VRAM the model is now loaded as the graph has raised up to roughly the 8GB limit; the size of the model being used for inference Llama 3.18B Instruct with my quantization settings

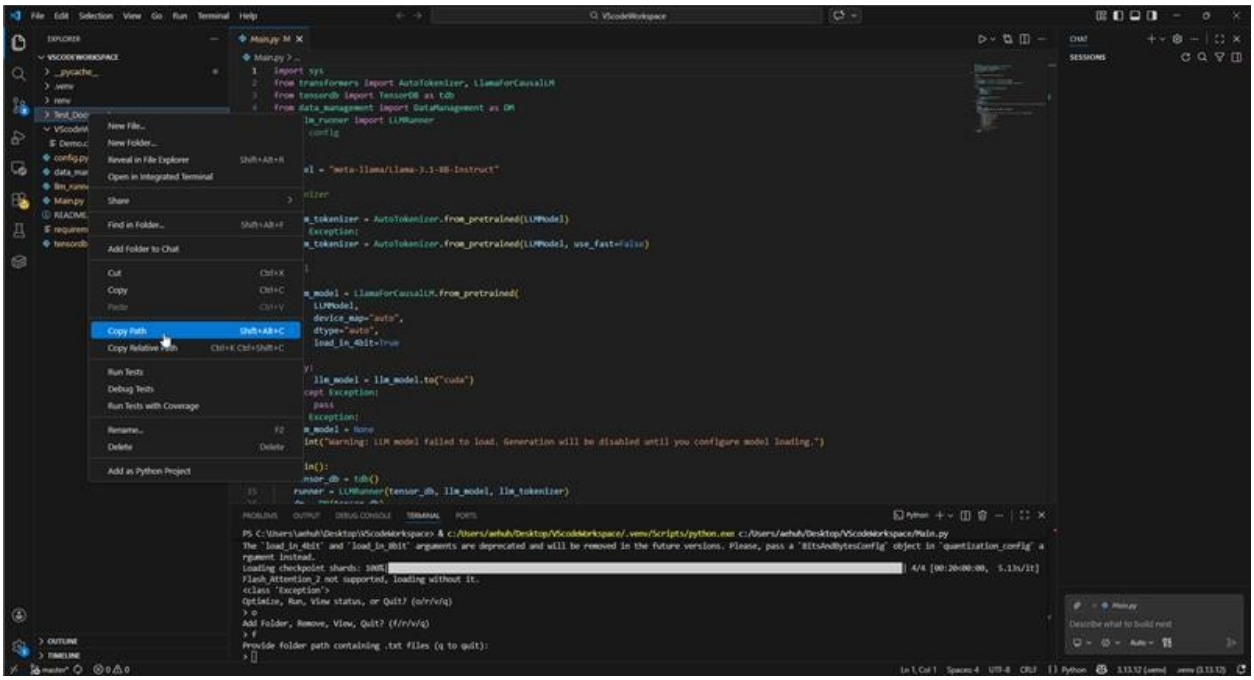
With the model loaded I now have to navigate through the various text prompts I created to navigate through the program options, first we need to add the documents to the document database so we enter O for optimize then F for add file

```
PS C:\Users\aehuh\Desktop\VScodeWorkspace> & c:/Users/aehuh/Desktop/VScodeWorkspace/.venv/Scripts/python.exe c:/Users/aehuh/Desktop/VScodeWorkspace/Main.py
  Please install 'en_core_web_lg' or 'en_core_web_trf' if you need NLP preprocessing.
  Please install 'en_core_web_lg' or 'en_core_web_trf' if you want NLP query processing.
  The `load_in_4bit` and `load_in_8bit` arguments are deprecated and will be removed in the future versions. Please, pass a `BitsAndBytesConfig` object in `quantization_config` a
  rgument instead.
  Loading checkpoint shards: 100%|                                                                                    | 4/4 [00:20<00:00,  5.13s/it]
  Flash_Attention_2 not supported, loading without it.
  <class 'Exception'>
  Optimize, Run, View status, or Quit? (o/r/v/q)
  >
```

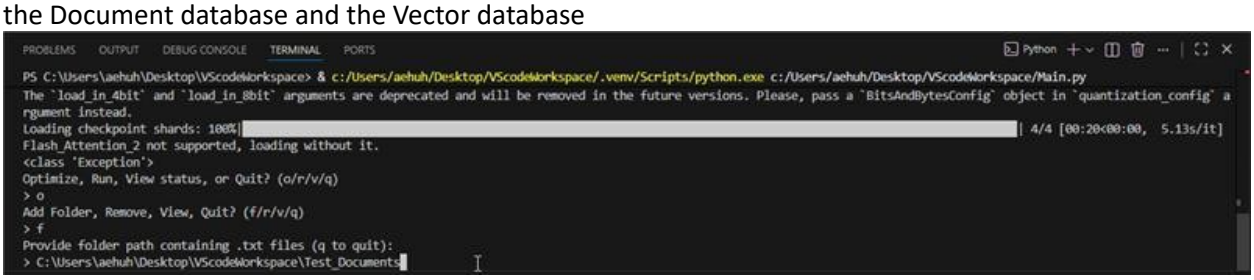
Now we need to go to the right hand side to find our files in Test_Documents



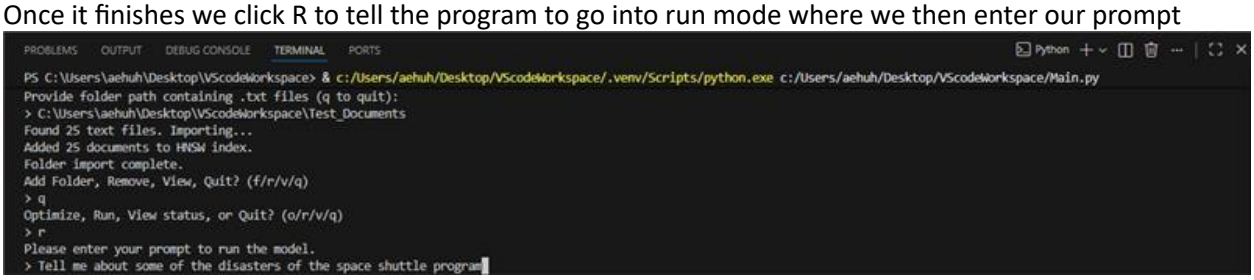
Next we copy path in order to find the path in my computer to the file so the python program can open it



We then paste it into the terminal so that the python program opens it up and loads the files into both the Document database and the Vector database



Once it finishes we click R to tell the program to go into run mode where we then enter our prompt



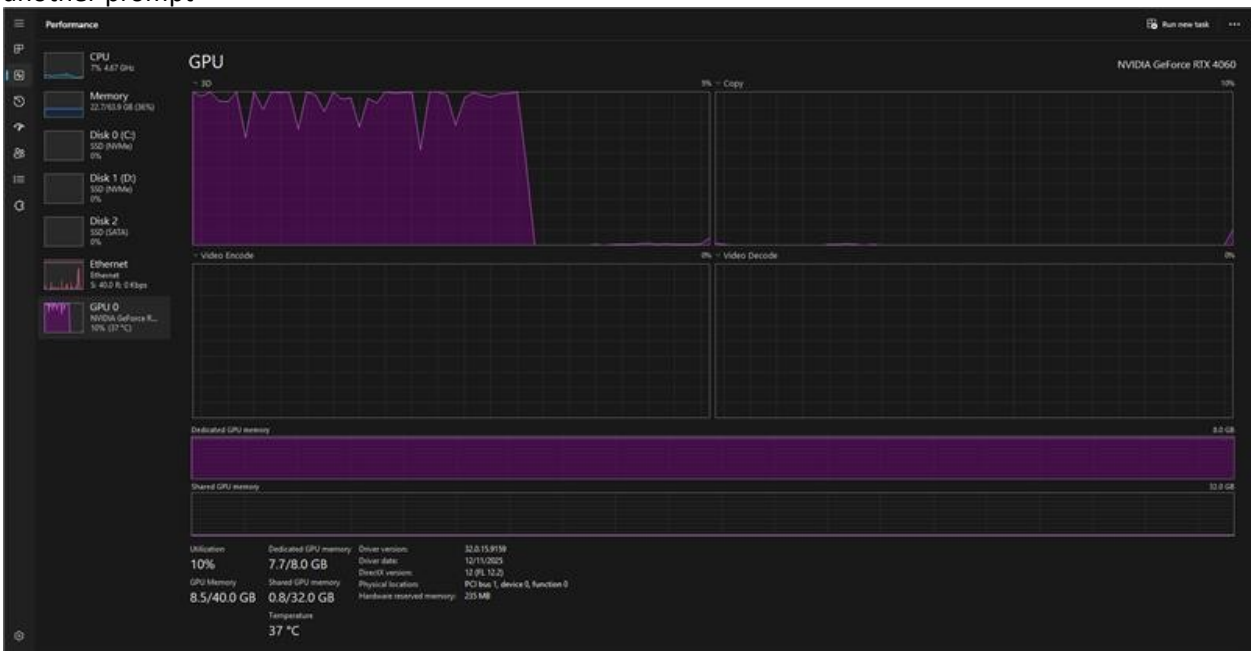
The program then creates and embedding and finds the two closest documents by semantic distance in the vector database, being documents 9 and 15, it then prints out the distances and one of the documents to double check the documents are relevant, it then begins to respond

As you can see in the GPU 3D usage category the model is maxing out the GPU's tensor calculations in order to compute the response

After about 30s of responding the model publishes its final response in relation to the prompt

```
Model output:
The Challenger disaster occurred on January 28, 1986. It was a major setback to the Space Shuttle program. The primary reason was a faulty O-ring that failed due to low tempera
tures during the launch, leading to a structural failure in the solid rocket boosters. The accident resulted in the deaths of seven crew members. The Challenger tragedy led to
a significant overhaul of safety protocols, ensuring more rigorous testing and inspection procedures for components.
The Columbia disaster happened on February 1, 2003. The main issue was a piece of foam insulation that broke off during the launch and damaged the shuttle's protective thermal
protection system. This damage allowed hot gases to penetrate and ignite the fuel tank, leading to the loss of the shuttle and its crew of seven. The Columbia disaster highligh
ted the risks of using a reusable spacecraft and led to a significant increase in safety measures, including more thorough inspections and testing of shuttle components and sys
tems.
Both disasters highlighted the importance of safety protocols and the risks associated with the reusable design of the space shuttles. They led to significant changes in NASA's
 safety procedures and the way the agency approaches safety in space exploration.
```

As you can see the model has now stopped is tensor core usage by the graph going to 0, however given the model is still loaded you can see that 7.7/8GB of VRAM is still being used to store it so it is ready for another prompt



Conclusion:

In conclusion I not only refreshed my programming skills but I also developed a number of crucial skills and built an understanding of the underlying infrastructure beneath many basic LLM implementations useful in SOHO environments where resources are limited but domain specific/company specific knowledge is